

Chapter 14: Forms

1. Overview

Forms are how Angular applications capture, validate, and synchronize user input with application state. Angular ships **two entirely different form architectures** that solve the same problem with opposite philosophies:

- **Template-driven forms** — the template is the source of truth. Directives (`ngModel`, `ngForm`, `ngModelGroup`) build a shadow `FormControl` / `FormGroup` tree behind the scenes, driven by two-way binding. You describe validation and structure declaratively in HTML.
- **Reactive forms** (a.k.a. model-driven forms) — the component class is the source of truth. You explicitly instantiate `FormControl`, `FormGroup`, and `FormArray` objects in TypeScript, and the template merely binds to that pre-built model via `formControlName` / `formGroup`.

Both sit on top of the same underlying abstract class, `AbstractControl`, and both rely on the `ControlValueAccessor` interface to bridge native DOM elements (or custom components) to the control model. Understanding forms deeply means understanding: how `AbstractControl` propagates value/status changes up and down a tree, how validators compose, how `ControlValueAccessor` wires the DOM, and where each API diverges in testability, scalability, and change-detection behavior.

This chapter also covers **typed reactive forms** (Angular 14+), which eliminate the historical `any`-typed weak spot of `FormGroup` / `FormControl`, and **dynamic forms**, where `FormArray` / `FormGroup` structures are built at runtime from metadata (e.g., a JSON schema driving a form).

Interviewers use forms as a proxy question for "do you understand Angular's change detection, RxJs, and component architecture in a real, non-toy scenario?" — so depth here pays off broadly.

2. Core Concepts

2.1 The `AbstractControl` Hierarchy

Everything descends from `AbstractControl`:

```
AbstractControl (abstract base: value, status, validators, errors, observables)
├── FormControl    - a single input's value/state
├── FormGroup      - a fixed set of named child controls (object-shaped)
└── FormArray     - an ordered list of child controls (array-shaped)
```

Every `AbstractControl` exposes:

- `value` — current value (aggregated for groups/arrays)
- `status` — `'VALID' | 'INVALID' | 'PENDING' | 'DISABLED'`
- `errors` — `ValidationErrors | null`, a dictionary like `{ required: true }`
- `valueChanges: Observable<T>` and `statusChanges: Observable<FormControlStatus>`
- State flags: `dirty/pristine`, `touched/untouched`, `valid/invalid`, `pending`

- Methods: `setValue()`, `patchValue()`, `reset()`, `markAsTouched()`, `markAsDirty()`, `disable()` / `enable()`, `updateValueAndValidity()`

`FormGroup` and `FormArray` are **composites** — their `value`, `status`, `errors` are derived from their children. A parent is `INVALID` if *any* descendant is invalid; a parent is `PENDING` if any descendant has an async validator running.

2.2 Template-Driven vs Reactive Forms — Decision Table

Dimension	Template-Driven	Reactive
Source of truth	Template (DOM/directives)	Component class (TypeScript)
Form model creation	Implicit, built by Angular via <code>NgModel</code> / <code>NgForm</code>	Explicit, via <code>FormControl</code> / <code>FormGroup</code> / <code>FormBuilder</code>
Module needed	<code>FormsModule</code>	<code>ReactiveFormsModule</code>
Data flow	Mutable, two-way (<code>[(ngModel)]</code>)	Immutable-ish, unidirectional; you push changes via <code>setValue</code> / <code>patchValue</code>
Validators	Directives in HTML (<code>required</code> , <code>minlength</code> , custom directive)	Functions passed in TS (<code>validators.required</code> , custom <code>validatorFn</code>)
Async validators	Directive-based, awkward	First-class (<code>asyncValidators</code> param)
Testability	Requires rendering the template (DOM-dependent, needs <code>fixture.detectChanges()</code> + <code>tick()</code>)	Pure unit tests — instantiate <code>FormGroup</code> , no TestBed/DOM needed
Dynamic forms (runtime-built)	Hard/unnatural (structure driven by template)	Natural fit — build <code>FormGroup</code> / <code>FormArray</code> programmatically
Scalability for complex forms	Poor — nested <code>ngModelGroup</code> , harder cross-field validation	Good — compose groups/arrays, custom validators at any level
Change detection cost	Higher — <code>NgModel</code> runs a digest-like check each CD cycle, uses <code>ngOnChanges</code> -driven diffing internally	Lower — direct subscription-based updates, no extra directive-level polling
Typed API (v14+)	Weakly typed (<code>ngModel</code> value is typically inferred loosely)	Strongly typed via generics (<code>FormControl<string></code> , <code>FormGroup<{...}></code>)
Best for	Simple forms, quick prototypes, forms mirroring a template 1:1	Complex forms, dynamic forms, forms needing rigorous testing/validation logic

Dimension	Template-Driven	Reactive
Boilerplate	Low upfront, more "magic"	More upfront setup, more explicit and traceable

Rule of thumb interviewers want to hear: template-driven forms are declarative and quick but hide the form model in directive internals, making testing and dynamic composition harder; reactive forms trade a bit of verbosity for full programmatic control, synchronous access to the model, and unit-testability without the DOM.

2.3 Template-Driven Forms — Mechanics

- `NgForm` is auto-attached to any `<form>` when `FormsModule` is imported (unless you opt out with `ngNoForm`).
- Each `[(ngModel)]` inside the form auto-registers an `NgModel` directive instance with the parent `NgForm`, creating a corresponding `FormControl` under the hood, keyed by the `name` attribute.
- `ngModelGroup="address"` creates a nested `FormGroup` named `address`.
- Validation is declared via *validator directives* (`required`, `minlength`, `pattern`, or a custom directive implementing `Validator` / `NG_VALIDATORS`).
- You get the underlying `FormGroup` via a template reference variable: `#f="ngForm"`, then `f.value`, `f.valid`, `f.form.get('email')`.

2.4 Reactive Forms — Mechanics

```
this.form = new FormGroup({
  email: new FormControl('', [Validators.required, Validators.email]),
  password: new FormControl('', Validators.required),
});
```

Or, more idiomatically, via `FormBuilder` (injectable service, less boilerplate, same result):

```
this.form = this.fb.group({
  email: ['', [Validators.required, Validators.email]],
  password: ['', Validators.required],
});
```

`FormBuilder.group()` accepts either a raw value, a `[value, validators, asyncValidators]` tuple, or an existing `AbstractControl`.

Template binds via `[formGroup]` and `formControlName`:

```
<form [formGroup]="form" (ngSubmit)="submit()">
  <input formControlName="email" />
  <input formControlName="password" type="password" />
</form>
```

There is **no two-way binding** here — the DOM element registers a `ControlValueAccessor` that both writes DOM→model (on input events) and model→DOM (via `writeValue`), but you never write `[(ngModel)]` syntax in a reactive form (mixing them is explicitly unsupported/discouraged).

2.5 Typed Forms (Angular 14+)

Before v14, `FormGroup.get('x')` returned `AbstractControl | null` with value typed as `any`, so typos in control names or type mismatches were only caught at runtime. Typed forms make the entire tree generic:

```
interface ProfileForm {
  name: FormControl<string>;
  age: FormControl<number>;
}

const form = new FormGroup<ProfileForm>({
  name: new FormControl('', { nonNullable: true }),
  age: new FormControl(0, { nonNullable: true }),
});

form.value.name;           // typed as string | undefined
form.getRawValue();        // { name: string; age: number } – includes disabled controls, non-optional
form.controls.name.setValue('Alice'); // compile error if you pass a number
```

Key typed-forms details:

- `nonNullable: true` prevents `reset()` from setting the control back to `null`; instead it resets to the initial value, and the control's type excludes `null`.
- Untyped/dynamic forms are still available via `FormGroup<any>` or `UntypedFormGroup` / `UntypedFormControl` for gradual migration.
- `FormBuilder` infers types automatically from initial values — `fb.group({ name: '' })` gives you `FormControl<string | null>` without manual annotation.
- `FormRecord<T>` is a typed variant for a `FormGroup` whose keys are dynamic (unknown at compile time) but whose value type is uniform — replaces the old pattern of casting a `FormGroup` to hold arbitrary keys.

2.6 Validators — Built-in, Custom, Async

Built-in synchronous validators (`validators` namespace): `required`, `requiredTrue`, `email`, `min`, `max`, `minLength`, `maxLength`, `pattern`, `nullValidator`. Compose multiple with `validators.compose(...)` (Angular does this automatically when you pass an array).

Custom synchronous validator — a pure function `ValidatorFn = (control: AbstractControl) => ValidationErrors | null`:

```
function forbiddenNameValidator(forbidden: RegExp): ValidatorFn {
  return (control: AbstractControl): ValidationErrors | null => {
    const forbidden_ = forbidden.test(control.value);
    return forbidden_ ? { forbiddenName: { value: control.value } } : null;
  };
}
```

Async validator — returns `Observable<ValidationErrors | null>` or `Promise<...>`, used for things requiring a server round-trip (e.g., "is this username taken?"):

```
function uniqueUsernameValidator(userService: UserService): AsyncValidatorFn {
  return (control: AbstractControl) =>
    control.value
      ? userService.checkUsername(control.value).pipe(
          map(isTaken => (isTaken ? { usernameTaken: true } : null)),
          catchError(() => of(null)),
        )
      : of(null);
}
```

Async validators run **after** all sync validators pass, and only when the control's status isn't already `INVALID`. While pending, `control.status === 'PENDING'`; `control.pending` is `true`. Angular debounces nothing for you by default — if you attach the async validator directly, it fires on every value change unless you set `updateOn: 'blur'` or manually debounce inside the validator with `debounceTime`.

Cross-field (group-level) validation — a validator attached to the `FormGroup` itself, not a single control, since it needs to compare sibling controls:

```
function passwordMatchValidator(group: AbstractControl): ValidationErrors | null {
  const pass = group.get('password')?.value;
  const confirm = group.get('confirmPassword')?.value;
  return pass === confirm ? null : { passwordMismatch: true };
}

this.form = this.fb.group(
  {
    password: ['', validators.required],
    confirmPassword: ['', validators.required],
  },
  { validators: passwordMatchValidator },
);
```

The resulting error lands on the **group**, not on either child control, so the template checks `form.errors?.['passwordMismatch']`, typically displayed near the confirm field, not bound to `confirmPassword.errors`.

2.7 `FormArray` for Dynamic Lists

`FormArray` models a variable-length list of controls (e.g., "add another phone number"):

```

get phones(): FormArray {
  return this.form.get('phones') as FormArray;
}

addPhone(): void {
  this.phones.push(this.fb.control('', validators.required));
}

removePhone(index: number): void {
  this.phones.removeAt(index);
}

```

```

<div formArrayName="phones">
  <div *ngFor="let phone of phones.controls; let i = index">
    <input [formControlName]="i" />
    <button (click)="removePhone(i)">Remove</button>
  </div>
</div>
<button (click)="addPhone()">Add phone</button>

```

Since Angular 14, `FormArray` also has a typed generic, `FormArray<FormControl<string>>`, and array-level validators (e.g., "at least one phone required") attach the same way as group-level validators.

3. Code Examples

3.1 Full Dynamic Reactive Form: Registration with Custom + Async Validation

```

// registration-form.component.ts
import { Component, OnDestroy, OnInit, inject } from '@angular/core';
import {
  FormArray,
  FormBuilder,
  FormControl,
  FormGroup,
  ValidationErrors,
  ValidatorFn,
  Validators,
} from '@angular/forms';
import { Subject, of } from 'rxjs';
import { catchError, debounceTime, distinctUntilChanged, map, switchMap, takeUntil } from 'rxjs/operators';
import { UserService } from '../user.service';

// --- Custom synchronous cross-field validator ---
function passwordsMatch(group: AbstractControl): ValidationErrors | null {
  const password = group.get('password')?.value;
  const confirm = group.get('confirmPassword')?.value;
  return password && confirm && password !== confirm ? { passwordMismatch: true } : null;
}

```

```

}

// --- Custom synchronous field validator ---
function nowhitespace(control: FormControl<string | null>): ValidationErrors | null {
  const value = control.value ?? '';
  return value.trim().length === 0 && value.length > 0 ? { whitespace: true } : null;
}

interface RegistrationForm {
  username: FormControl<string>;
  password: FormControl<string>;
  confirmPassword: FormControl<string>;
  skills: FormArray<FormControl<string>>;
}

@Component({
  selector: 'app-registration-form',
  templateUrl: './registration-form.component.html',
})
export class RegistrationFormComponent implements OnInit, OnDestroy {
  private readonly fb = inject(FormBuilder);
  private readonly userService = inject(UserService);
  private readonly destroy$ = new Subject<void>();

  form!: FormGroup<RegistrationForm>;

  ngOnInit(): void {
    this.form = this.fb.group(
      {
        username: this.fb.control('', {
          nonNullable: true,
          validators: [Validators.required, validators.minLength(4)],
          asyncValidators: [this.uniqueUsernameValidator()],
          updateOn: 'blur', // avoid firing async validator on every keystroke
        }),
        password: this.fb.control('', {
          nonNullable: true,
          validators: [Validators.required, validators.minLength(8)],
        }),
        confirmPassword: this.fb.control('', {
          nonNullable: true,
          validators: [Validators.required, nowhitespace],
        }),
        skills: this.fb.array<FormControl<string>>(
          [this.fb.control('', { nonNullable: true, validators: validators.required })],
          { validators: this.minArrayLength(1) },
        ),
      },
      { validators: passwordsMatch },
    );

    // React to overall status changes (e.g., to toggle a submit button, log analytics)
    this.form.statusChanges

```

```

        .pipe(distinctUntilChanged(), takeUntil(this.destroy$))
        .subscribe(status => console.log('form status:', status));
    }

    get skills(): FormArray<FormControl<string>> {
        return this.form.controls.skills;
    }

    addSkill(): void {
        this.skills.push(this.fb.control('', { nonNullable: true, validators:
validators.required }));
    }

    removeSkill(index: number): void {
        this.skills.removeAt(index);
    }

    private minArrayLength(min: number): ValidatorFn {
        return (control: AbstractControl) =>
            (control as FormArray).length >= min ? null : { minArrayLength: { min } };
    }

    private uniqueUsernameValidator() {
        return (control: FormControl<string>) =>
            of(control.value).pipe(
                debounceTime(300),
                switchMap(value => this.userService.isUsernameTaken(value)),
                map(taken => (taken ? { usernameTaken: true } : null)),
                catchError(() => of(null)), // never let a failed HTTP call block the form as
invalid
            );
    }

    submit(): void {
        if (this.form.invalid) {
            this.form.markAllAsTouched(); // surface validation errors for untouched controls
            return;
        }
        console.log(this.form.getRawValue());
    }

    ngOnDestroy(): void {
        this.destroy$.next();
        this.destroy$.complete();
    }
}

```

```

<!-- registration-form.component.html -->
<form [formGroup]="form" (ngSubmit)="submit()">
  <label>
    Username
    <input formControlName="username" />

```



```

    <span *ngIf="form.controls.username.pending">Checking availability...</span>
    <span *ngIf="form.controls.username.errors?.['required'] &&
form.controls.username.touched">
      Username is required.
    </span>
    <span *ngIf="form.controls.username.errors?.['usernameTaken']">
      Username already taken.
    </span>
  </label>

  <label>
    Password
    <input type="password" formControlName="password" />
  </label>

  <label>
    Confirm Password
    <input type="password" formControlName="confirmPassword" />
  </label>
  <span *ngIf="form.errors?.['passwordMismatch'] && form.controls.confirmPassword.touched">
    Passwords do not match.
  </span>

  <div formArrayName="skills">
    <div *ngFor="let skill of skills.controls; let i = index">
      <input [formControlName]="i" />
      <button type="button" (click)="removeSkill(i)">Remove</button>
    </div>
  </div>
  <button type="button" (click)="addSkill()">Add skill</button>
  <span *ngIf="skills.errors?.['minArrayLength']">At least one skill is required.</span>

  <button type="submit" [disabled]="form.pending">Register</button>
</form>

```

3.2 A Minimal Custom ControlValueAccessor

```

@Component({
  selector: 'app-star-rating',
  template: `
    <span *ngFor="let s of [1,2,3,4,5]" (click)="rate(s)">{{ s <= value ? '★' : '☆' }}
  </span>
  `,
  providers: [
    {
      provide: NG_VALUE_ACCESSOR,
      useExisting: forwardRef(() => StarRatingComponent),
      multi: true,
    },
  ],
})
export class StarRatingComponent implements ControlValueAccessor {

```

```

value = 0;
private onChange: (value: number) => void = () => {};
private onTouched: () => void = () => {};

rate(value: number): void {
  this.value = value;
  this.onChange(value);
  this.onTouched();
}

writeValue(value: number): void { this.value = value ?? 0; }
registerOnChange(fn: (value: number) => void): void { this.onChange = fn; }
registerOnTouched(fn: () => void): void { this.onTouched = fn; }
setDisabledState?(isDisabled: boolean): void { /* toggle disabled UI state */ }
}

```

Now `<app-star-rating formControlName="rating">` works exactly like a native input.

4. Internal Working

4.1 `ControlValueAccessor` — the DOM/model bridge

`ControlValueAccessor` (CVA) is the interface that lets Angular's forms module talk to *any* UI element — native `<input>`, `<select>`, or a custom component — without knowing its internals. It defines four members:

```

interface ControlValueAccessor {
  writeValue(obj: any): void;
  registerOnChange(fn: any): void;
  registerOnTouched(fn: any): void;
  setDisabledState?(isDisabled: boolean): void;
}

```

For native elements, Angular ships built-in accessors (`DefaultValueAccessor` for text inputs, `SelectControlValueAccessor`, `CheckboxControlValueAccessor`, `NumberValueAccessor`, `RadioControlValueAccessor`) registered against `NG_VALUE_ACCESSOR` with `multi: true`. When `formControlName / ngModel` is applied to an element, Angular's `NgControl` directive machinery (specifically the `FormControlName / NgModel` directive constructor) injects `NG_VALUE_ACCESSOR` and, from that multi-provider array, **picks the one accessor whose host element matches** (falling back to `DefaultValueAccessor` for a plain `<input>`).

The wiring flow:

1. **Model → DOM:** When the `FormControl`'s value is set programmatically (`setValue`, `reset`, initial value), the directive calls `accessor.writeValue(newValue)`, which the CVA implementation uses to update the native DOM property (e.g., `input.value = ...`) or component state.
2. **DOM → Model:** The CVA registers, via `registerOnChange(fn)`, a callback that Angular's forms module hands it. The CVA implementation invokes this callback whenever the underlying DOM fires its "value changed" event (e.g., a `(input)` listener for text, `(change)` for checkboxes). Calling this callback

updates the `FormControl`'s value and re-runs validation, which then propagates status/value changes up the tree.

3. **Touched state:** `registerOnTouched(fn)` supplies a callback the CVA calls on blur (`blur`) event, which marks the control as `touched`.
4. **Disabled state:** `setDisabledState(isDisabled)` is called whenever `control.disable() / enable()` runs, letting the CVA toggle the native `disabled` attribute or component-level disabled flag.

This is precisely why a custom component becomes form-compatible just by implementing CVA and registering itself as an `NG_VALUE_ACCESSOR` provider — Angular's `FormControlName` directive doesn't care whether it's talking to an `<input>` or a `<app-star-rating>`; it only ever calls the four CVA methods.

4.2 `NgModel` and Change Detection

`NgModel` implements `ControlValueAccessor` consumption itself (it is an `NgControl` implementation, not a CVA) and creates a `FormControl` under the hood via its constructor. On each change detection cycle:

- `NgModel` implements `onChanges`, so if the bound expression (`[ngModel]="someValue"`) changes as an `@Input`, Angular calls `ngOnChanges`, and `NgModel` calls `writeValue` on the associated CVA to push the new value into the DOM.
- The CVA's registered `onChange` callback (wired to a native `input / change` DOM event listener) fires **outside** the normal CD input-binding cycle — it's a plain DOM event handler. When it fires, it updates the internal `FormControl.value`, which emits on `valueChanges`, and also calls `NgModel`'s internal `viewToModelUpdate()`, which emits the `ngModelChange @Output()`. Because `[(ngModel)]` desugars to `[ngModel]="x" (ngModelChange)="x=$event"`, this is how "two-way binding" is actually implemented — it's really one-way input binding plus an output event, banana-in-a-box syntax.
- Since the DOM event triggers Angular's zone (`NgZone.onMicrotaskEmpty / event listener patch`), a full change detection pass is scheduled afterward, at which point the updated value re-renders anywhere else it's bound.

4.3 Value/Status Propagation up a `FormGroup` Tree

Each `AbstractControl` node maintains its own `_parent` reference. When a leaf `FormControl`'s value changes:

1. The control runs its own validators synchronously, sets `this.errors`, computes its own `status`.
2. It emits on its own `valueChanges` and `statusChanges` Subjects.
3. It calls `this._parent?.updateValueAndValidity()` (unless `{ onlySelf: true }` was passed), which recomputes the **parent's** aggregate value (by reading all children's values into an object/array) and aggregate status:
 - Parent status is `INVALID` if any child is `INVALID`.
 - Parent status is `PENDING` if any child is `PENDING` (and no child is `INVALID`).
 - Otherwise the parent runs its *own* group-level/array-level validators (e.g., `passwordsMatch`) against its own aggregated value.
4. The parent then emits its own `valueChanges / statusChanges`, and recursively calls **its** parent's `updateValueAndValidity()` — this is how a change three levels deep in a nested `FormGroup` bubbles all the way up to the root form's `valid / invalid / value`.

This is a **bottom-up push** model for status/value; conversely, `setValue / patchValue / reset / disable` called on a parent perform a **top-down push**, distributing values down to matching child controls (recursively calling `setValue` on each child), then bubbling the final aggregate status back up once.

`valueChanges` and `statusChanges` are implemented as `EventEmitter` (RxJS `Subject` subclass), so they're **hot observables** — a late subscriber only gets future emissions, and Angular does not multicast the *current* value to a new subscriber automatically (unlike `BehaviorSubject`). This is why patterns like `form.get('x')!.valueChanges.pipe(startWith(form.get('x')!.value))` are common when you need the "current value" as the first emission.

5. Edge Cases & Gotchas

- **ExpressionChangedAfterItHasBeenCheckedError from form state:** If a component's template reads `form.valid / control.errors` and you programmatically call `setValidators()` + `updateValueAndValidity()` (or `markAsTouched()`) inside `ngAfterViewInit` or in response to a `valueChanges` subscription that fires *during* the same CD cycle, Angular may detect that a previously-checked binding (e.g., `[disabled]="form.invalid"`) now yields a different value in dev mode's second check pass. Fix: perform such state mutations in `ngOnInit` (before first CD pass) or wrap in `Promise.resolve().then(() => ...) / setTimeout / ChangeDetectorRef.detectChanges()` to defer to the next cycle.
- **updateOn strategies ('change' | 'blur' | 'submit'):** By default every control uses `'change'` — validity/value updates on every keystroke, which is expensive for async validators (fires an HTTP call per keystroke unless debounced) and can cause premature "invalid" flicker on partially-typed input. `updateOn: 'blur'` defers both value propagation and validation until the blur event; `updateOn: 'submit'` defers until form submission. This can be set per-control or for an entire `FormGroup` (inherited by children unless overridden). **Gotcha:** setting `updateOn: 'blur'` on a `FormGroup` does **not** automatically make children use `'blur'` unless the child also specifies it or omits its own `updateOn` (children inherit the parent's default only if unset) — mixed strategies inside one form are easy to get wrong and hard to spot in review.
- **Memory leaks from valueChanges / statusChanges subscriptions:** These are long-lived `Subject`s owned by the control, not automatically torn down when the *component* is destroyed (only when the control itself is garbage collected, which usually only happens if the form is discarded). Any manual `.subscribe()` inside a component **must** be unsubscribed in `ngOnDestroy` (via `takeUntil(this.destroy$)`, `takeUntilDestroyed()` in newer Angular, or the `async` pipe) — otherwise you leak the subscription (and anything it closes over) for the app's lifetime, and worse, callbacks keep firing against a destroyed component, sometimes writing to now-stale DOM references.
- **Disabled controls are excluded from value / getRawValue() differs:** A disabled `FormControl` is omitted from the parent's `.value` object entirely (and from the emitted `submit` / HTTP payload if you naively serialize `form.value`), but `validators.required` still won't fire on it and it's excluded from validity aggregation. Use `form.getRawValue()` to get the *full* value including disabled controls when you need it (e.g., pre-filling a PATCH payload).
- **FormArray index-based binding breaks on reorder:** Because `formControlName / [formControlName]="i"` binds by array index, removing or reordering items via drag-and-drop can cause the wrong control to bind to the wrong DOM element unless you use `trackBy`

correctly in `*ngFor` — track by the control instance (`trackBy: (i, control) => control`) rather than by index.

- **patchValue vs setValue:** `setValue` requires the exact shape (all keys) and throws if you omit one — a deliberate safety net to catch typos in dynamic forms. `patchValue` silently ignores missing keys, which is convenient but can hide bugs (e.g., a typo'd key like `'usernaem'` just never gets applied, no error).
- **Mixing template-driven and reactive directives** (`[(ngModel)]` inside a `[formGroup]`) is explicitly unsupported — Angular throws `NG01350` ("ngModel cannot be used to register form controls with a parent formGroup directive") unless you opt in with the deprecated `ngModelOptions.standalone: true` to detach it from the reactive form.
- **Async validator debounce responsibility is yours:** Angular does not debounce async validators for you; if you don't set `updateOn: 'blur'` or add `debounceTime` inside the validator pipeline, a fast typist can fire many concurrent HTTP validation requests, and responses can race — always ensure the validator's Observable uses `switchMap` (not `mergeMap`) so a newer request cancels the older in-flight one.

6. Interview Questions & Answers

Q1. What's the fundamental difference between template-driven and reactive forms?

A: Template-driven forms build the `FormControl / FormGroup` model implicitly, driven by directives in the template (`ngModel`, `ngForm`) — the template is the source of truth. Reactive forms have you explicitly construct the `FormControl / FormGroup / FormArray` tree in the component class, and the template merely binds to it — the class is the source of truth. This affects testability (reactive forms can be unit tested without touching the DOM), dynamic form construction (much easier in reactive), and validation style (directives vs. functions).

Q2. Why would you choose reactive forms over template-driven forms for a complex form?

Interviewer intent: Checking whether the candidate has actually built non-trivial forms, not just copy-pasted a tutorial.

A: Complex forms typically need: dynamic structure (fields added/removed at runtime, e.g., `FormArray` for repeatable line items), cross-field validation, async validation with debouncing, and rigorous unit tests independent of rendering. Reactive forms handle all of these naturally because the form model is a plain object graph you can construct, mutate, and test in isolation. Template-driven forms couple the model to the DOM structure, making dynamic composition and pure unit testing much harder.

Q3. What is `AbstractControl` and how do `FormControl`, `FormGroup`, and `FormArray` relate to it?

A: `AbstractControl` is the abstract base class providing the shared API — `value`, `status`, `errors`, `valueChanges`, `statusChanges`, `dirty / touched` flags, and validity methods. `FormControl` represents a single value/leaf. `FormGroup` composes named child controls into an object-shaped value. `FormArray` composes ordered child controls into an array-shaped value. Both `FormGroup` and `FormArray` derive their own `value / status` from aggregating their children's values/statuses.

Q4. How do you write a custom synchronous validator, and what must it return?

A: A `validatorFn` is a pure function (`control: AbstractControl => ValidationErrors | null`). It must return `null` if valid, or an object like `{ errorKey: errorDetails }` if invalid. You attach it via the control's validators array (`validators.compose` under the hood combines multiple). Example: a "no whitespace" validator checks `control.value.trim().length === 0` and returns `{ whitespace: true }` if so.

Q5. How is an async validator different from a sync validator, and when does it run?

A: An `AsyncValidatorFn` returns an `Observable<ValidationErrors | null>` (or a `Promise`). It only runs **after** all synchronous validators pass (if any sync validator fails, the async ones are skipped entirely). While the async validator's observable hasn't emitted, the control's status is `PENDING`. You typically use async validators for server-side checks (username availability, uniqueness) and must debounce/cancel with `switchMap` to avoid race conditions from rapid input.

Q6. How does cross-field validation work, e.g., validating that "password" and "confirmPassword" match?

A: You cannot validate this on either individual `FormControl` since neither has access to its sibling. Instead, you attach a validator to the parent `FormGroup` itself (via the second argument to `fb.group()` or `group.setValidators()`), which receives the group as its `control` argument and can call `group.get('password') / group.get('confirmPassword')`. The resulting error lives on the group (`form.errors?.['passwordMismatch']`), not on either child control.

Q7. What do `dirty`, `pristine`, `touched`, and `untouched` mean, and how do they differ from `valid/invalid`?

A: `pristine/dirty` track whether the control's **value** has ever been changed by the user (programmatic `setValue` also marks dirty unless you pass `{ emitEvent: false }`... actually note: `setValue` does mark dirty by default only via user interaction paths — precise answer: dirty/pristine reflects value-change history, not validity). `touched/untouched` track whether the control has ever received and lost focus (blur). `valid/invalid` are purely about whether the current value passes validators — completely orthogonal axes. A field can be `valid` and `pristine` (untouched, still passes because empty isn't required), or `invalid` and `dirty` (user typed something wrong). UI patterns typically show error messages only when `invalid && (dirty || touched)`, to avoid yelling at users before they've interacted with the field.

Q8. What is `ControlValueAccessor` and why does Angular need it?

Interviewer intent: Tests whether the candidate understands forms below the surface API — a strong signal of real depth.

A: `ControlValueAccessor` is the interface that decouples the forms module from any specific UI element. It defines `writeValue` (model→DOM), `registerOnChange / registerOnTouched` (DOM→model callbacks), and `setDisabledState`. Angular ships built-in CVAs for native elements (text inputs, checkboxes, selects, radios). To make a custom component (e.g., a star-rating widget or a custom date picker) usable with `formControlName / ngModel`, you implement `ControlValueAccessor` yourself and register it as an `NG_VALUE_ACCESSOR` multi-provider — this is what lets `FormControlName` treat your component exactly like a native `<input>`.

Q9. How does a value change in a nested `FormControl` propagate up to the root `FormGroup`'s `valid` status?

A: Each `AbstractControl` holds a `_parent` reference. When a leaf control's value changes, it recomputes its own status/errors, emits `valuechanges / statuschanges`, then calls `_parent.updateValueAndValidity()`. The parent group aggregates all children's values into its own `.value`, sets its own status to `INVALID` if any child is invalid (or runs its own group-level validators if all children are individually valid), emits its own `valuechanges / statuschanges`, and recursively notifies *its* parent. This bubbles all the way to the root, which is why checking `rootForm.valid` reflects the state of every nested control.

Q10. What's the difference between `setValue` and `patchValue`?

A: `setValue` requires you to provide a value for every control in the group/array's exact shape; it throws an error if a key is missing or extraneous, which catches typos and shape mismatches early. `patchValue` accepts a partial object and only updates the keys you provide, silently ignoring the rest — convenient for partial updates (like applying a subset of fields from an API response) but can silently hide a typo'd key.

Q11. What are typed reactive forms and what problem do they solve?

Interviewer intent: Tests whether the candidate has kept up with Angular 14+ changes, not just legacy knowledge.

A: Before Angular 14, `FormGroup` / `FormControl` were implicitly typed as `any`, so `form.get('email')` (a typo) compiled fine and returned `null` at runtime, and `form.value.email` gave no compile-time type safety. Typed forms make `FormControl<T>`, `FormGroup<{...}>`, and `FormArray<T>` generic, so the compiler catches wrong control names, wrong value types in `setValue`, and gives autocomplete on `.controls`. The `nonNullable: true` option ensures `reset()` doesn't reintroduce `null` into the type. Old behavior remains available via `UntypedFormControl` / `UntypedFormGroup` for incremental migration.

Q12. How would you build a form where the number of fields isn't known until runtime (e.g., driven by a JSON schema)?

A: Use reactive forms with `FormArray` / `FormGroup` built programmatically: iterate the schema/metadata, and for each field definition call `this.fb.control(initialValue, validatorsForThatField)`, pushing controls into a `FormArray` or assigning them as keys on a `FormGroup` constructed with `new FormGroup({})` and then `.addControl(name, control)` for each. The template uses `*ngFor` over `Object.keys(form.controls)` or `formArray.controls`, binding generically (e.g., a switch on field `type` to render `<input>`, `<select>`, checkbox, etc., each bound via `[formControlName]`). This is essentially impossible to do cleanly with template-driven forms since directives require field names to exist statically in the template.

Q13. Why does Angular throw `ExpressionChangedAfterItHasBeenCheckedError` in some form-related scenarios, and how do you avoid it?

A: It happens when a template binding that depends on form state (e.g., `[disabled]="form.invalid"`) evaluates to a different value on Angular's second dev-mode verification pass than it did on the first, typically because a lifecycle hook or a synchronous `valueChanges` callback mutates validators/state (e.g., calling `setValidators` + `updateValueAndValidity()`, or `markAsTouched()`) *after* the initial CD pass already read the old value in the same cycle. Fixes: perform such mutations in `ngOnInit` before the first check, or defer them to the next macro/microtask (`setTimeout`, `Promise.resolve().then()`), or trigger an explicit `ChangeDetectorRef.detectChanges()` afterward.

Q14. What's the danger of subscribing to `valueChanges` without unsubscribing, and how do you avoid it idiomatically?

A: `valueChanges` / `statusChanges` are `Subject`-backed observables owned by the control, not the component — they outlive the component if not cleaned up, and continue invoking your subscription callback (which may reference destroyed component state or trigger unwanted side effects like duplicate API calls) after the component is gone, and each un-cleaned subscription is a retained closure — a classic memory leak. Idiomatic fixes: pipe through `takeUntil(this.destroy$)` (emit/complete `destroy$` in `ngOnDestroy`), use the newer `takeUntilDestroyed()` (Angular 16+, ties to `DestroyRef` automatically), or prefer the `async` pipe in the template wherever the value is just for display, since it auto-unsubscribes on view destruction.

Q15. When would a `FormGroup`'s status be `PENDING`, and how does that affect a submit button?

A: `PENDING` occurs while at least one descendant control's async validator has an in-flight, unresolved observable/promise (and no descendant is already `INVALID`, since sync-invalid short-circuits async validation). Since `form.valid` is `false` while `status === 'PENDING'` (only `'VALID'` counts as valid), a `[disabled]="form.invalid"` submit button will correctly stay disabled during the async check — but for good UX you usually distinguish the two states explicitly (`form.pending` to show a spinner/"checking..." message vs. `form.invalid` to show a hard validation error), rather than lumping them into one generic "disabled" message.

Q16. What happens if you mix `[(ngModel)]` inside a form that also has `[formGroup]` applied?

A: Angular throws a runtime error (`NG01350`) because `NgModel` tries to register itself as a child control of the parent `FormGroup` directive, but reactive and template-driven control registration mechanisms are mutually exclusive — you can't have both an implicit (template-driven) and explicit (reactive) control simultaneously map to the same DOM element under one form directive. To use `ngModel`-style two-way binding for a field *outside* the reactive model (e.g., a UI-only toggle not part of the submitted form data), you must add `[ngModelOptions]="{standalone: true}"` to detach that particular `ngModel` from the parent `FormGroup`.

7. Quick Revision Cheat Sheet

- **Template-driven** = template is source of truth, directives (`ngModel`, `ngForm`) build the model implicitly; needs `FormsModule`; good for simple forms, poor for dynamic/testable ones.
- **Reactive** = class is source of truth, you build `FormControl` / `FormGroup` / `FormArray` explicitly; needs `ReactiveFormsModule`; good for complex, dynamic, and unit-testable forms.
- `AbstractControl` base class → `FormControl` (leaf), `FormGroup` (named children), `FormArray` (ordered children). Parent status/value = aggregate of children.
- `FormBuilder` (`fb.group/control/array`) = convenience factory, same result as `new FormGroup(...)`.
- **Validators:** sync `validatorFn` returns `ValidationErrors | null`; async `AsyncValidatorFn` returns `Observable/Promise<ValidationErrors | null>` and only runs after sync validators pass; group-level validators enable cross-field checks (attach to the `FormGroup`, not a child).
- **State flags:** `dirty/pristine` = value ever changed; `touched/untouched` = ever blurred; `valid/invalid` = passes validators — three independent axes; UI usually gates error display on `invalid && (dirty || touched)`.
- `FormArray` = dynamic-length lists (add/remove controls at runtime: `push`, `removeAt`, `insert`).
- **Typed forms (v14+):** `FormControl<T>`, `FormGroup<{...}>`, `nonNullable: true` avoids `null` reintroduction on `reset()`; `UntypedForm*` for legacy/gradual migration.
- `ControlValueAccessor`: the DOM↔model bridge — `writeValue` (model→DOM), `registerOnChange` / `registerOnTouched` (DOM→model), `setDisabledState`; register custom components via `NG_VALUE_ACCESSOR` multi-provider to make them form-compatible.
- **Propagation:** leaf change → validates itself → emits `valueChanges` / `statusChanges` → calls `_parent.updateValueAndValidity()` → bubbles to root (bottom-up); `setValue` / `disable` on a parent push down to children first (top-down).
- `setValue` = strict, full shape required; `patchValue` = partial, silently ignores unknown/missing keys; `getRawValue()` = includes disabled controls' values (unlike `.value`).

- **Gotchas:** `ExpressionChangedAfterItHasBeenCheckedError` from validator/state mutations after first CD pass; `updateOn: 'blur' | 'submit'` to throttle expensive (especially async) validation; always unsubscribe `valueChanges / statusChanges` (`takeUntil / takeUntilDestroyed / async` pipe) to avoid leaks; never mix `ngModel` and `formGroup` on the same control without `standalone: true`.

Created By - Durgesh Singh